

# CSCI 210: Computer Organization

## Lecture 10: Control Flow

Stephen Checkoway

Slides from Cynthia Taylor

# Announcements

- Problem Set 2 due today

# CS History: The If-Else Statement

- Haskell Curry and Willa Wyatt are the first people to describe performing different instructions based on the result of a previous calculation, on the Eniac in 1946
- Early assembly language instructions jumped to a new memory location based on a specific condition, were not general purpose
- Fortran (1957) specifying jumps to three locations at once, depending on whether a calculation was negative, zero, or positive, and gave it the name "if."
- Flow-matic (Grace Hopper, 1958), used comparisons between numbers and used the name "otherwise" for else
- In 1958, a German computing organization proposed an if statement that took an arbitrary Boolean statement, had an "else" case, and returned control to immediately after the if/else statement after completing the statement

# Today: Program control flow

- High level languages have many ways to control the order of execution in a program: if, if-else, for loops, while loops
- Today we will look at how these higher order concepts are built out of RISC-V control flow instructions

# Control Flow

- Recall the basic instruction cycle
  - $IR = \text{Memory}[PC]$
  - $PC = PC + 4$
- Both branch and jump instructions change the value of the program counter

# Control Flow - Instructions

- Conditional branch instructions
  - `beq, bne, blt, bge, bltu, bgeu`: compare two registers and change the value of the program counter (pc) if the comparison (`=`, `≠`, `<`, or `≥`) is true
  - `bltu` and `bgeu` perform unsigned comparisons
  - Pseudoinstructions `bgt[u]`, and `ble[u]` by reversing operands
- Unconditional jumps
  - `jal, jalr`: jump to a location
  - Always change the value of the program counter
  - Pseudoinstructions `call, j, jr`, and `ret`

# Control Flow - Labels

- In assembly, we use labels—human readable names—to help us guide control flow. Labels can be the target of branch or jump instructions.

- Example:

```
    beq    rs1, rs2, label
```

```
    ...
```

```
label:
```

```
    add    t1, t0, t2
```

- Assemblers are responsible for translating labels into addresses (or offsets from the address of the instruction to the labeled address)

# Branch instruction usage

- Branch instructions are used to implement high-level programming concepts like if statements and loops
- ```
if (x < y) {  
    // body of the "then" branch  
}
```
- To implement this, we need to branch over the body of the then branch if the condition is **false**

High-level code

```
if (x < 0)
    x = y + z;
```

Assuming x, y, and z are integers in registers t0, t1, and t2, respectively, which are the equivalent assembly instructions?

A

```
        blt t0, zero, Label
Label:  add t0, t1, t2
```

B

```
        blt t0, zero, Label
        add t0, t1, t2
Label:  ...
```

C

```
        bge t0, zero, Label
        add t0, t1, t2
Label:  ...
```

D – None of these is correct.

# If-else statements

```
if (condition) {  
    // body of then branch  
} else {  
    // body of else branch  
}
```

- If the condition is false, we'll branch to the body of the else
- If the condition is true, we'll execute the body of the then branch but we need a way to **jump** over the body of the else

# Jump instructions

- Jump and link (register) instructions

```
jal    rd, label
```

```
jalr   rd, rs
```

set the pc to the address of the instruction at label (or in `rs`)

- The address of the instruction following `jal/jalr` is placed in register `rd`, we'll return to this next class
- For now, let's focus on the pseudoinstruction

```
j      label    # equiv. to jal zero, label
```

which just sets the pc and doesn't change any other registers

[why doesn't it set any other registers?]

```
if (x < y)
    z = x;
else
    z = y;
```

Assuming `x`, `y`, and `z` are integers in registers `t0`, `t1`, and `t2`, respectively. Which are the equivalent assembly instructions?

**A**

```
bge    t0, t1, else
mv     t2, t0
else:  mv     t2, t1
```

**B**

```
blt    t0, t1, else
mv     t2, t0
j      endif
else:  mv     t2, t1
endif:
```

**C**

```
bge    t0, t1, else
mv     t2, t0
j      endif
else:  mv     t2, t1
endif:
```

**D – None of the above**

# Loops

```
while (condition) {  
    // body  
}  
for (x = 0; x < 20; x += 1) {  
    // body  
}
```

- We can rewrite `for` loops as `while` loops
- For `while` loops, negate the condition and branch over the body of the loop
- After the body of the loop, unconditionally jump back to the top of the loop where the condition is checked

# Translating a for loop to RISC-V

```
for (i = 0; i < 10; i += 1) {  
    print_int(i);  
}
```

```
.text  
main:  
    li      t0, 0  
    li      t1, 10  
  
loop:  
    bge     t0, t1, end  
    # print int syscall  
    li      a7, 1  
    mv      a0, t0  
    ecall  
    addi    t0, t0, 1  
    j      loop  
  
end:
```

# How to access elements of an array in a loop

- Can't programmatically change the offset in  
`ld rd, offset(rs)`
- Need to change the *base address* instead
- Assume elements in the array are  $n$  bytes in size
- Two options
  1. Compute  $\text{index} * n$  and add that to the address of the start of the array
  2. Start with a pointer set to the start of the array and increment it by  $n$  each time through the loop

```

long A[100];
// ...
for (long i = 0; i < 100; i += 1) {
    A[i] = i;
}

```

- The address of A is placed in s0
- t0 is the index variable i
- t1 is the upper bound on the loop
- t0 is shifted left by 3 to multiply it by 8 and then s0 is added (option 1)
- t0 is stored in memory at 0 (t2)
- t0 is incremented at the end of the loop

```

.data
# array of 100 doublewords
A:
.space 800
.text
main:
    la      s0, A
    li      t0, 0
    li      t1, 100

loop:
    bge     t0, t1, end
    # t2 = t0 * 8 + s0
    slli    t2, t0, 3
    add     t2, t2, s0
    # A[t0] = t0
    sd      t0, 0(t2)
    addi    t0, t0, 1
    j       loop

end:

```

```

.data
# array of 100 doublewords
A:
.space 800
.text
main:
    la    s0, A
    li    t0, 0
    li    t1, 100

loop:
    bge   t0, t1, end
    # t2 = t0 * 8 + s0
    slli  t2, t0, 3
    add   t2, t2, s0
    # A[t0] = t0
    sd    t0, 0(t2)
    addi  t0, t0, 1
    j     loop

end:

```

In the previous example (shown to the left) the loop was

```

for (i = 0; i < 100; i += 1)
    A[i] = i;

```

Now consider the same loop but with body

```
A[i] = 0;
```

In groups, discuss how the assembly can be simplified for this new loop (use fewer instructions) by using option 2:

- “Start with a pointer set to the start of the array and increment it by the size of each element each time through the loop”

Select any clicker answer when your group is done

# Solution

```
.data
# array of 100 doublewords
arr:
.space 800
.text
main:
    la      s0, arr
    addi    t1, s0, 800

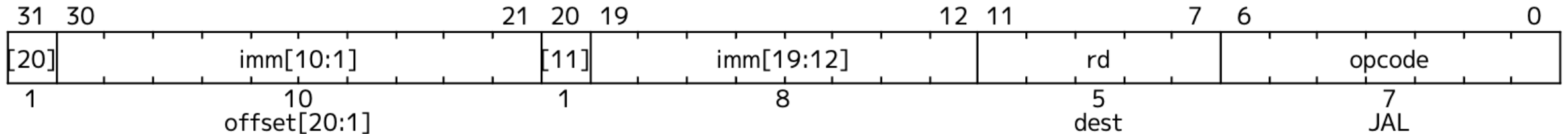
loop:
    bge     s0, t1, end
    sd      zero, 0(s0)
    addi    s0, s0, 8
    j       loop

end:
```

## Key changes

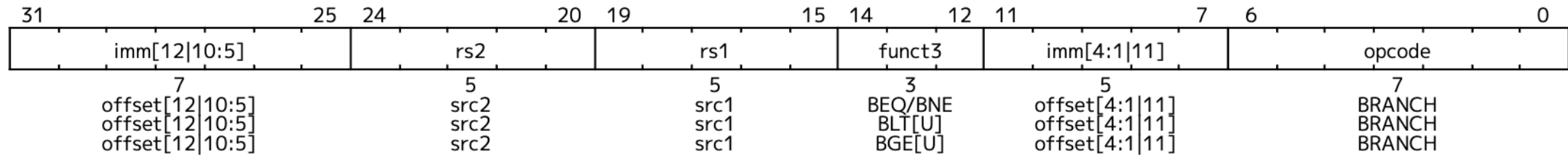
- Eliminated the index variable in `t0`
- Changed the loop bound to `s0 + 800`
- Incremented the base address (`s0`) by 8 rather than the the index variable by 1

# Jump instruction encoding



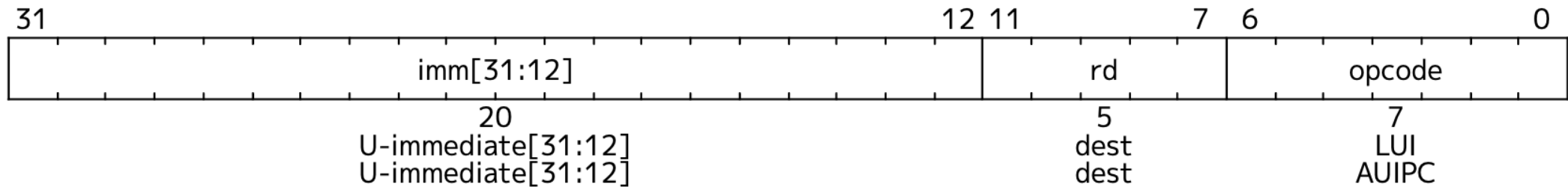
- Jump and link instruction `j al rd, offset` (the real instruction corresponding to the `j` pseudoinstruction) uses a new instruction format: J-type
- This encoding is **bonkers!**
- A 21-bit offset from the address of the `j al` itself to the target is encoded, in order from left-to-right in the instruction as bits 20, 10–1, 11, 19–12
- Note that bit 0 of the offset isn't encoded because it's always 0
- Bit 1 is *also* always zero in the version of RISC-V we're looking at but it gets encoded anyway
- Upshot: `j al` (and thus `j`) can jump anywhere with  $\pm 1$  MB of the `j al` instruction
- Longer jumps can be made using the `j alr` instruction

# Branch instruction encoding



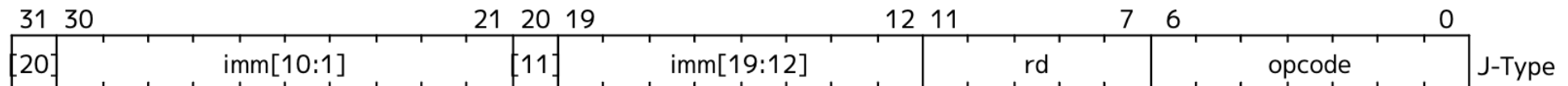
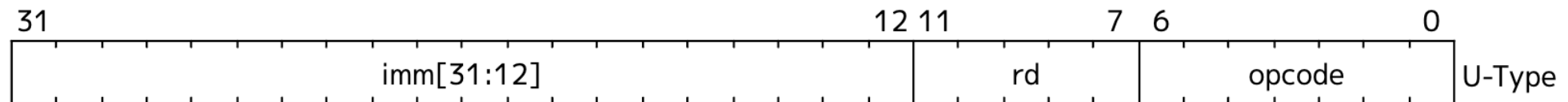
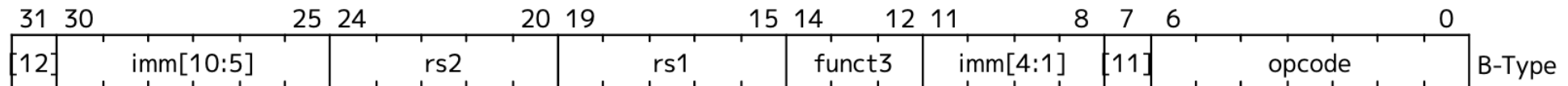
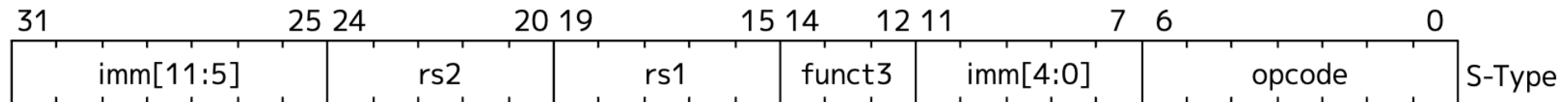
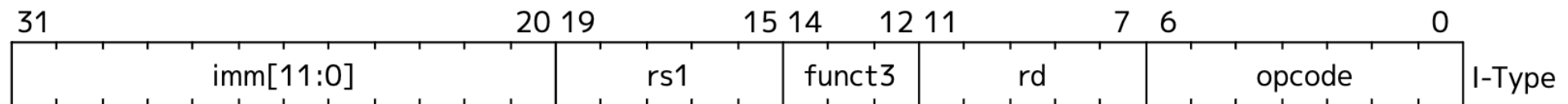
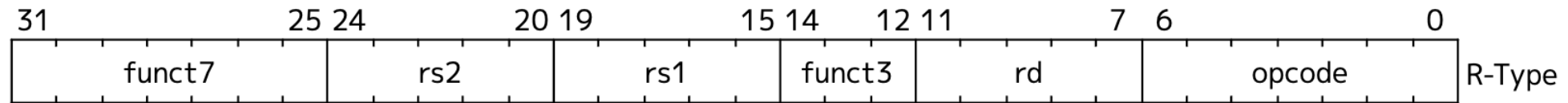
- Branch instructions are encoded as the new B-type format
- B-type is *almost* identical to the S-type (store instructions)
- A 13-bit branch target offset is encoded in 12 bits (the LSB of the offset is always 0) and thus not encoded
- The second-most-significant bit of the offset (bit 11) is encoded in the least-significant bit of the 5-bit immediate field

# The final instruction format: U-type



- Load upper immediate (`lui`) and add upper immediate pc (`auipc`) use the final instruction format: U-type
- `lui rd, imm` **sets** `rd = sign-extend(imm << 20)`
- `auipc rd, imm` **sets**  
`rd = pc + sign-extend(imm << 20)`
- These are typically paired with `addi` or `jalr` (next class) to form 32-bit immediates or offsets from pc

# All base RISC-V instruction formats



The textbook uses the old names SB and UJ for B-type and J-type formats

# Reading

- Next lecture: Procedures
  - Section 2.8
- Problem set 2: Due tonight
- Lab 2: Available